Reporte Técnico de Actividades Práctico-Experimentales Nro. 06

1. Datos de identificación del estudiante y la práctica

Nombre del estudiante(s)	Maria Soledad Buri Camacho
Asignatura	Simulacion
Ciclo	5
Unidad	U2: Generación de números pseudoaleatorios y generación de variables aleatorias
Resultado de aprendizaje de la unidad	R2. Interpreta los resultados de una simulación bajo principios de transparencia, responsabilidad y honestidad.
Práctica Nro.	06
Título de la Práctica	Validación estadística de números pseudoaleatorios mediante Chi-cuadrado y Kolmogorov-Smirnov
Nombre del Docente	José O. Guamän Q.
Fecha	05 de Noviembre de 2026 06 de Noviembre 2026
Horario	Jueves 10h30 a 11h30 Viernes 07h30 a 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3

2. Objetivo(s) de la práctica

Evaluar la calidad estadística de los números pseudoaleatorios generados mediante el método congruencial lineal (LCG), utilizando las pruebas Chi-cuadrado y Kolmogorov-

Smirnov para analizar su compatibilidad con la distribución uniforme en el intervalo [0, 1).

- Generar secuencias pseudoaleatorias mediante LCG en Python y Java.
- Aplicar Chi-cuadrado para comparar frecuencias observadas y esperadas.
- Aplicar Kolmogorov-Smirnov para comparar la distribución empírica con la teórica.
- Interpretar los estadísticos de prueba, valores p y decisiones de H_0.
- Comparar diferentes configuraciones del generador y los resultados de ambos lenguajes.

3. Materiales, reactivos, equipos y herramientas

- Computador con sistema operativo Linux.
- Python 3, bibliotecas SciPy y Matplotlib.
- Java y JDK para compilación y ejecución.
- Editor de código y archivos CSV para el registro de resultados.

4. Procedimiento / Metodología Ejecutada

Para el desarrollo de la práctica se siguieron las siete etapas de un proyecto de simulación, tomando como base los procedimientos de la guía Nro. 006.

Etapa 1. Definición del problema

- Se planteó comprobar si las secuencias generadas mediante el método LCG presentan un comportamiento compatible con una distribución uniforme entre 0 y 1.

Etapa 2. Formulación conceptual

- Se definió el modelo LCG: $X(n+1) = (a \cdot X(n) + c) \bmod m$ y $U(n) = X(n)/m$. Se establecieron las hipótesis H_0: $X \sim U(0,1)$ y H_1: X no sigue $U(0,1)$, con $\alpha = 0,05$.

Etapa 3. Recolección y análisis de datos

- Se utilizaron 10 000 observaciones, semilla 2026 y diez intervalos. Se compararon dos configuraciones: buena ($a = 1664525$, $c = 1013904223$, $m = 2^{32}$) y mala ($a = 5$, $c = 1$, $m = 16$). Estos datos son parámetros del experimento, no mediciones externas.

Etapa 4. Construcción del modelo

- Se programó el generador en los archivos generador.py y Generador.java. El análisis estadístico se implementó en practica06.py y Practica06.java, manteniendo separados generación y análisis.

Etapa 5. Verificación y validación

- Se comprobaron los primeros estados y números normalizados en Python y Java; también se compararon sumas de control para distintos tamaños. Se aplicó Chi-cuadrado con $\chi^2 = \Sigma[(O_i - E_i)^2/E_i]$, $E_i = n/10$ y $gl = 9$; y KS con $D = \sup|F_n(x) - F(x)|$.

Etapa 6. Experimentación y análisis de resultados

- Se ejecutaron ambas configuraciones, se calcularon los valores p de Chi-cuadrado y KS y se compararon con 0,05. Además, se elaboraron gráficas de frecuencias y distribuciones acumuladas, y se midieron los tiempos de ejecución.

Etapa 7. Implementación y documentación

• Se guardaron las secuencias y resultados en archivos CSV, se generaron gráficas PNG y se organizó el código con un README para Linux. Por último, se elaboró este reporte técnico.

5. Resultados

Resultados del generador LCG

Se generaron 10 000 números por configuración, con semilla 2026. Se utilizaron los mismos parámetros en Python y Java para permitir la comparación.

Indicador	Configuración buena	Configuración mala
Semilla	2026	2026
Cantidad de números	10 000	10 000
Multiplicador (a)	1 664 525	5
Incremento (c)	1 013 904 223	1
Módulo (m)	2^{32} = 4 294 967 296	16

Comparación de las primeras observaciones

Posición	Estado X_n	Python U_n	Java U_n
1	91264577	0.021249190	0.021249190
2	190675628	0.044395129	0.044395129
3	165328411	0.038493520	0.038493520
4	2847667390	0.663024231	0.663024231
5	2769032453	0.644715608	0.644715608
6	2078868128	0.484024204	0.484024204

Los primeros valores coinciden entre los dos lenguajes porque se aplica la misma fórmula LCG con idéntica semilla y parámetros. El archivo comparar.py también registró coincidencia de los checksums en los tamaños evaluados.

Resultados de Chi-cuadrado y Kolmogorov-Smirnov

Se aplicó un nivel de significancia de $\alpha = 0,05$. Si $p \geq 0,05$ no se rechaza H_0; si $p < 0,05$, se rechaza. Las frecuencias esperadas fueron 1 000 por intervalo y los grados de libertad de Chi-cuadrado fueron 9.

Configuración	Lenguaje	χ^2	p Chi²	D (KS)	p KS
Buena	Python	7,520	0,583145	0,008520	0,459774
Buena	Java	7,520	0,583145	0,008520	0,460726
Mala	Python	937,500	$5,133 \times 10^{-196}$	0,062500	$2,113 \times 10^{-34}$
Mala	Java	937,500	$5,133 \times 10^{-196}$	0,062500	$1,948 \times 10^{-34}$

En la configuración buena, ambos valores p superan 0,05, por lo que no se rechaza la hipótesis de uniformidad. En la configuración mala, los valores p son inferiores a 0,05 y H_0 se rechaza en ambas pruebas. La pequeña diferencia entre los valores p de KS se debe a los distintos procedimientos de cálculo utilizados en Python y Java.

Representación gráfica de los resultados

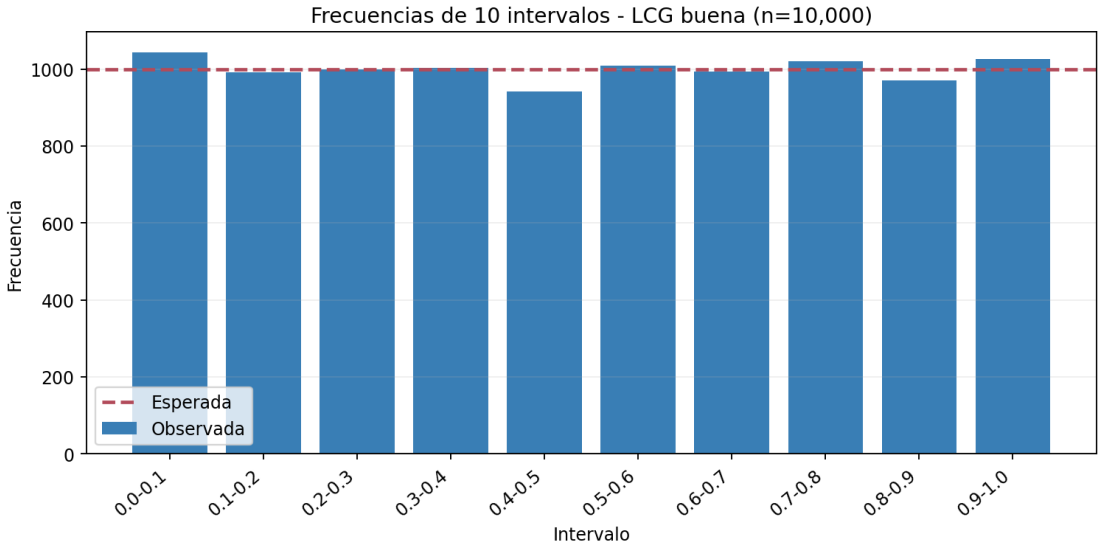


Figura 1. Frecuencias observadas y esperadas: configuración buena.

Las barras se mantienen alrededor de la frecuencia esperada de 1 000; no se observan diferencias suficientemente grandes para rechazar uniformidad con Chi-cuadrado ($\chi^2 = 7,52$).

Comparación complementaria Python y Java

La tabla corresponde a la configuración buena.

N	π estimado	Error abs.	Python (ms)	Java (ms)
1 000	3,136000	0,005593	0,223	0,123
10 000	3,129600	0,011993	2,451	0,930
100 000	3,141040	0,000553	23,537	5,899
1 000 000	3,144040	0,002447	268,291	15,527

En estas ejecuciones Java registró tiempos menores que Python y ambos produjeron la misma estimación de π con iguales parámetros. Los tiempos dependen del equipo y de las condiciones de medición; estimar π es una actividad complementaria, no una prueba de uniformidad.

6. Preguntas de Control

1. ¿Qué es la hipótesis nula?

- Es la afirmación inicial que se somete a evaluación estadística. En esta práctica, H_0 establece que la secuencia generada es compatible con una distribución uniforme U(0,1).

2. ¿Qué representa el valor p?

- Es la probabilidad, suponiendo verdadera H_0, de obtener un estadístico tan extremo o más que el observado. Permite decidir si existe evidencia suficiente para rechazar la hipótesis nula; no mide la probabilidad de que H_0 sea verdadera.

3. ¿Qué significa rechazar H_0?

- Significa que los resultados presentan una diferencia estadísticamente significativa respecto de la distribución uniforme para $\alpha = 0,05$. Esto ocurrió con nuestra configuración mala en las dos pruebas.

4. ¿Qué diferencia existe entre Chi-cuadrado y KS?

- Chi-cuadrado agrupa los números en diez intervalos y compara las frecuencias observadas con las esperadas; KS utiliza los datos ordenados para medir la máxima diferencia entre la distribución acumulada empírica y la teórica.

5.¿Por qué una prueba estadística no demuestra que un generador sea verdaderamente aleatorio?

- Porque solo evalúa ciertas propiedades sobre una muestra finita. Un LCG es determinista y puede presentar patrones o dependencias que las pruebas de uniformidad no detectan; por ello, superar ambas pruebas no garantiza aleatoriedad verdadera.

7. Conclusiones

- Se implementó el generador congruencial lineal en Python y Java, separando en ambos lenguajes la generación de los números del análisis estadístico.
- La prueba Chi-cuadrado mostró diferencias entre las configuraciones: la buena alcanzó $\chi^2 = 7{,}52$ y la mala $\chi^2 = 937{,}50$, con $n = 10\,000$ y $gl = 9$.
- La prueba Kolmogorov-Smirnov confirmó la diferencia: se obtuvo $D = 0{,}00852$ en la configuración buena y $D = 0{,}06250$ en la mala.

8. Recomendaciones

- Ejecutar las pruebas con otras semillas y tamaños de muestra para observar la estabilidad de los resultados.
- Utilizar parámetros justificados en el LCG y aplicar también pruebas de independencia antes de emplear las secuencias en simulaciones más complejas.

9. Reflexión Critica

- La práctica permitió comprender que obtener números entre 0 y 1 no significa automáticamente que estén distribuidos de manera uniforme. Mediante Chi-cuadrado y KS fue posible identificar diferencias que no siempre se aprecian a simple vista. También se comprobó que organizar el código en módulos facilita su revisión y que la interpretación de los valores p es tan importante como su cálculo.

10. Anexos

Link del GitHub:

https://github.com/Zoe777zz/Practica_06_Simulacion_5A.git

Declaración de uso de IA Generativa:

1. Herramienta de Inteligencia Artificial utilizada

- ChatGPT, desarrollada por OpenAI.

2. Forma en que se utilizó la herramienta

- Se utilizó como apoyo en la organización del proyecto, estructuración de los archivos Python y Java, revisión de las fórmulas, interpretación de las pruebas Chi-cuadrado

y KS, elaboración de gráficas y redacción del informe. Los resultados incluidos proceden de las ejecuciones y los archivos de salida del proyecto.

3. Prompts usados

- **Prompt 1:** Ayúdame a implementar las pruebas estadísticas Chi-cuadrado y Kolmogorov-Smirnov en Python y Java para validar la uniformidad de números pseudoaleatorios generados mediante el método LCG.